Planejamento ServerRest

1. Informações Gerais

Este documento define o Plano de Testes para a atividade da semana 10, seguindo uma abordagem simplificada e focada nos requisitos essenciais da norma ISO/IEC/IEEE 29119-3 facilitada.

1.1. Identificação do Projeto/Sistema

- **Nome do Projeto/Sistema:** Planejamento de Teste - API ServerRest
- **Data de Criação do Plano:** 02 de fev. de 2026
- **Elaborado por:** Igor Herculano Silva
- **API testada:**

ServeRest
compassuol.serverest.dev

1.2. Objetivos e Escopo dos Testes

Este plano foca em reconhecer e validar as regras de negocio a partir dos dados documentos no Swagger.

- **Objetivos dos Testes:**
  - Verificar **endpoints**
  - Verificar o **fluxo de valor**
  - Verificar funcionamento dos **tokens**
  - Treinar Técnicas de testes exploratórios, testes funcionais, engenharia de qualidade e planejamento de testes..

1.3. Escopo (Por modulo)

Usuário	Login	Produto	Carrinho
Cadastro (POST)	Geração de **Token**	Cadastro (POST)	Cadastro (POST)
Busca,listagem (GET)	-	Busca, listagem (GET)	Busca, listagem (GET)
Edição (PUT)	-	Edição (PUT)	Exclusão (DELETE)
Exclusão (DELETE)	-	Exclusão (DELETE)	

1.4. Fora do escopo

- Testes de **Carga** e **Stress**
- **Interface**
- **Infra** e **Banco de Dados**
- Testes de usabilidade da **Documentação** ou da **Lógica de Negócios**

2. Abordagem de Teste

A abordagem de teste será manual e exploratória, focada na validação de contratos, códigos de status HTTP e lógica de negócios, utilizando ferramentas de requisição (**Postman**) e a documentação **Swagger.**.

2.1. Estratégia de Teste

A execução dos testes exploratórios será gerenciada através da metodologia de **Gestão de Testes Baseada em Sessão** (SBTM). As sessões serão focadas em validar os endpoints, manipulando o JSON de entrada (payload) para verificar a robustez da API e a integridade dos dados.

2.2. Tipos de Teste

- **Testes de Contrato:** Validação da estrutura do JSON de resposta e Tipagem de dados.
- **Testes Funcionais:** Verificação das regras de negócio (CRUD) e Status Codes (200, 201, 400, 401).
- **Testes de Segurança Básica:** Validação de acesso a rotas protegidas sem token ou com token inválido.

3. Ambiente de Teste

3.1. Necessidades de Hardware e Software

Tipo	Especificação
Servidor de Aplicação	Ambiente Externo de Produção
Estações de Teste	Windows 11, Postman
Documentação	Swagger

3.2. Dados de Teste

Os dados do teste serão de uma conta real criada para fins de teste.

4. Critérios de entrada

4.1. Critérios de entrada

O ciclo de testes só poderá iniciar quando:

- A API estiver acessível publicamente na URL `compassuol.serverest.dev/`
- As *Collections* do Postman estiverem criadas com as variáveis de ambiente configuradas (`base_url`).
- O ambiente no Jira/QAlity estiver organizado para criação de ciclos e bugs.
- A documentação (Swagger) estiver disponível para consulta.
- O documento de Plano de Testes estiver finalizado e entregue na plataforma de ensino (Learning).

4.2. Critérios de saída

- 100% dos Cenários de Teste planejados (Seção 6) forem executados.
- Todos os defeitos Críticos e Altos encontrados estiverem devidamente reportados no Jira com evidências.
- A taxa de realização dos testes de "Caminho Feliz" (Reconhecimento da API) for 100%.

5. Gerenciamento de Teste

5.1. Gerenciamento de Casos de Teste

O gerenciamento do ciclo de vida dos testes será realizado na ferramenta **Jira**, utilizando o plugin **QAlity** para estruturação e execução.

- **Repositório:** Os cenários de teste serão criados no QAlity e vinculados às Histórias de Usuário para garantir rastreabilidade.
- **Execução:** A execução técnica ocorre no Postman, e o status (Pass/Fail) é atualizado no "Test Cycle" do QAlity.

5.2. Gerenciamento de Não Conformidades (Defeitos)

Os defeitos encontrados serão catalogados no **Jira** como *Bugs*, seguindo rigorosamente o template abaixo:

Template de Report de Bug:

6. Cenários de teste funcionais

ID	Pilar (VADER)	Cenário de Teste	Objetivo Técnico	Resultado Esperado
CT-01	Verbs	Enviar PUT na rota /login	Validar se a API bloqueia verbos não suportados.	Status 405 Method Not Allowed.
CT-02	Verbs	Enviar GET na rota /login	Verificar se credenciais trafegam via URL.	Status 405 ou 400.
CT-03	Authorization	Tentar cadastrar Produto sem enviar Header Authorization	Validar barreira de entrada.	**Status 401** Unauthorized; Msg: "Token de acesso ausente".
CT-04	Authorization	Tentar cadastrar Produto com Token de usuário comum	Validar RBAC.	**Status 403** Forbidden; Msg: "Rota exclusiva para administradores".
CT-05	Authorization	Manipular Token e tentar POST /produtos	Testar integridade da assinatura do Token.	**Status 401** Unauthorized.

CT-06	Data	Cadastrar usuário com e-mail inválido	Validar sanitização e formato de dados.	**Status 400** Bad Request.
CT-07	Data	Cadastrar produto com preço negativo	Validar regras de negócio e integridade financeira.	**Status 400** Bad Request.
CT-08	Data	Cadastrar usuário enviando campo administrador como booleano true	Validar tipagem estrita.	**Status 400** (Falha de validação de schema).
CT-09	Data	Enviar Payload vazio {} no POST /usuarios	Validar obrigatoriedade de campos.	**Status 400** Bad Request; Listar campos obrigatórios faltantes.
CT-10	Errors	Tentar excluir usuário com ID inexistente	Verificar tratamento de recurso não encontrado.	**Status 200** (Mensagem: "Nenhum registro excluído") ou **400**.
CT-11	Errors	Enviar JSON malformado	Verificar se a API trata parse error sem expor stack trace.	**Status 400** Bad Request.
CT-12	Errors	Tentar cadastrar e-mail já existente	Verificar mensagem de erro de duplicidade.	**Status 400** Msg: "Este email já está sendo usado".
CT-13	Responsiveness	Medir tempo de resposta do login	Garantir UX aceitável.	**Status 200** Tempo < (**Número a determinar**)
CT-14	Responsiveness	Listar produtos (GET /produtos)	Verificar performance de consulta ao banco.	**Status 200** Tempo < (**Número a determinar**)

7. Riscos

Risco (Falha Provável)	Probabilidade	Impacto	Exemplo possíveis
Falso Positivo em Autenticação	**Média**	**Crítico**	A API permitir acesso a rotas de Admin usando token de usuário comum.
Inconsistência de Dados (Lixo no Banco)	**Média**	**Médio**	A API aceitar cadastro de produtos com preço negativo ou nomes vazios.
Persistência Volátil	**Alta**	**Baixo**	O servidor reiniciar e apagar os usuários criados no teste anterior.

8. Mapeamento de Endpoints

Módulo	Método	Endpoint	Objetivo Principal
Login	**POST**	/login	Autenticar usuário e gerar Token.
Usuários	**GET**	/usuarios	Listar todos os cadastros.
	POST	/usuarios	Cadastrar novo usuário.
	DELETE	/usuarios/{_id}	Excluir usuário.
Produtos	**GET**	/produtos	Listar produtos disponíveis.
	POST	/produtos	Cadastrar novo produto (Requer Token).
	POST	/carrinhos	Criar carrinho com produtos.

Carrinhos	**DELETE**	/carrinhos/concluir-compra	Finalizar compra e limpar carrinho.
	DELETE	/carrinhos/cancelar-compra	Cancelar compra e limpar carrinho.

9. Exemplos de requisição

1. Login

Endpoint: `/login`

POST (Autenticação)

JSON

```
{
  "email": "fulano.qa@teste.com.br",
  "password": "teste"
}
```

---

2. Usuários

Endpoint: `/usuarios`

POST (Cadastro de Usuário)

JSON

```
{
  "nome": "Igor QA Teste",
  "email": "igor.qa.teste1@teste.com.br",
  "password": "teste",
  "administrador": "true"
}
```

PUT (Edição de Usuário) *URL:* `/usuarios/{_id}`

JSON

```
{
  "nome": "Igor QA Editado",
  "email": "igor.qa.teste1@teste.com.br",
  "password": "nova_senha_123",
  "administrador": "true"
}
```

---

3. Produtos

Endpoint: `/produtos` *Requisito: Header* `Authorization: Bearer <seu_token>`

POST (Cadastro de Produto)

JSON

```
{
  "nome": "Notebook Gamer",
  "preco": 12000,
  "descricao": "Notebook para jogos de alta performance",
  "quantidade": 10
}
```

PUT (Edição de Produto) *URL:* `/produtos/{_id}`

JSON

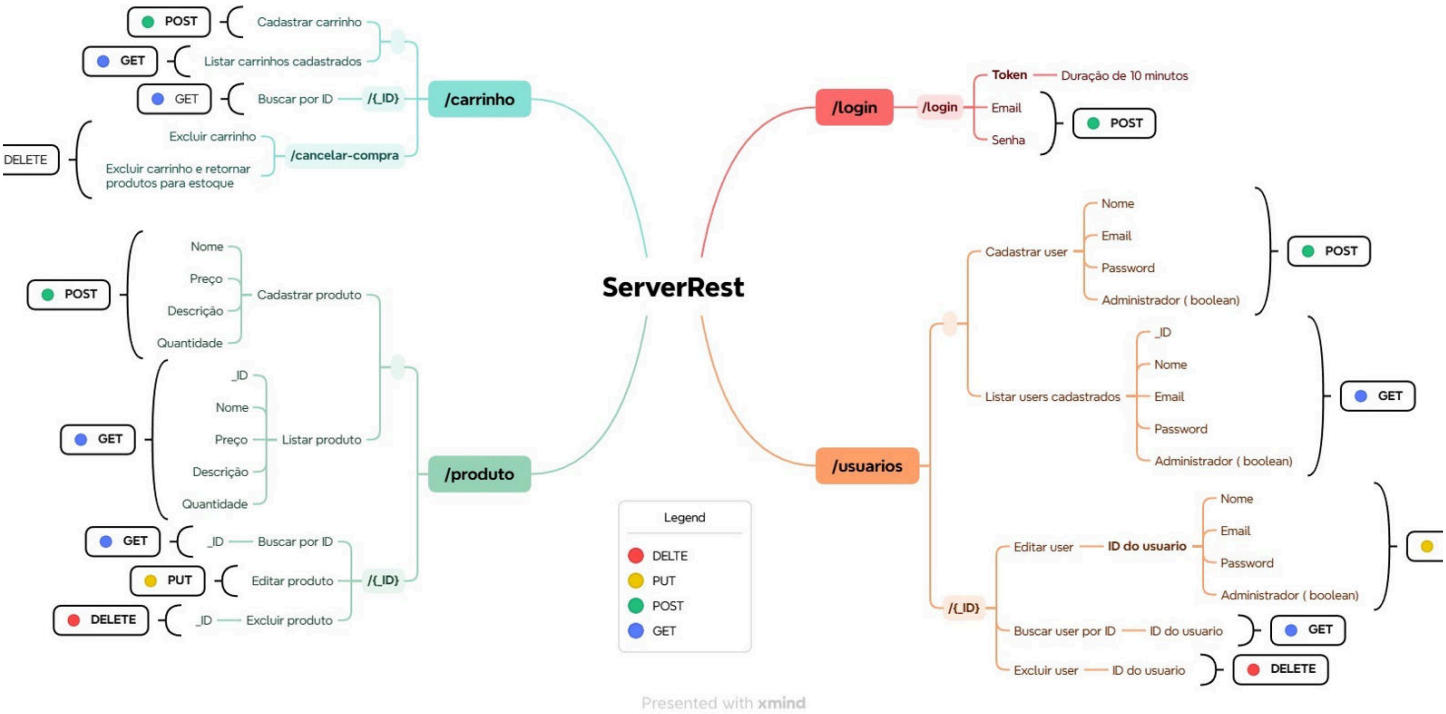
```
{
  "nome": "Notebook Gamer
  V2",
  "preco": 11500,
  "descricao": "Versão atualizada com mais RAM",
  "quantidade": 15
}
```

4. Carrinhos

Endpoint: `/carrinhos`

POST (Criar Carrinho)

JSON

```
{
  "produtos": [
    {
      "idProduto": "BeeJh5lz3k6kSIzA",
      "quantidade": 1
    },
    {
      "idProduto": "K6leHdftCeOjj8BJ",
      "quantidade": 2
    }
  ]
}
```

10. Mapa Mental